

# Fixing the wording of parenthesized aggregate-initialization

## The problem

The status quo wording fails to implement the design intent of P0960; right now, the wording does not allow single-argument parenthesized initialization, function-style casts, `static_casts`, or C-style casts if the conversion's target type is an aggregate. The parenthesized initialization creates a special case where parenthesized initialization with multiple arguments is well-formed, but where parenthesized initialization with one argument is ill-formed.

In addition to conveniently making library facilities such as `make_shared` Just Work with aggregates, the whole point of P0960 is to remove special cases. The validity of parenthesized initialization should not (and in fact must not) depend on the number of arguments (unless there are more arguments than there are elements to initialize, of course).

Note that the avoidance of special cases doesn't mean special cases in the technical wording; it means special cases in how to write programs.

Some arguments have been made that `static_casting` from fewer arguments than there are elements is unfortunate. However,

1. that's a pre-existing problem, not something introduced by P0960
2. it seems very unwise to use an existing inconsistency and an aggregate vs. non-aggregate compatibility problem to protect against the arguably unfortunate aspects of what casts do.

P0960 wasn't designed with a specific goal of making `static_casts` work; however, it was designed with the specific goal that if braced-init is valid, paren-init should be valid, and by extension, if one kind of initialization is valid, other kinds of initialization should also be valid. Prior to P0960, parenthesized aggregate initialization was valid for some cases (copy/move), and its intent was to remove the warts where a form of initialization was not valid solely because the target type is an aggregate. Aggregates are supposed to be simple and straightforward; prior to P0960, the special-case rules strongly recommended users to avoid those special cases by turning their aggregates into non-aggregates by adding constructors, which is very much counter-productive.

## Acknowledgements

An anonymous contributor, known to be an expert in both The Queen's English and forging iron, helpfully provided the wording for this fix.

## Wording

## Change in [expr.static.cast]p4:

An expression *e* can be explicitly converted to a type *T* if there is an implicit conversion sequence (12.4.3.1) from *e* to *T*, ~~or~~ if overload resolution for a direct-initialization (9.4) of an object or reference of type *T* from *e* would find at least one viable function (12.4.2), or if *T* is an aggregate type ([dcl.init.aggr]) having a first element *x* and there is an implicit conversion sequence from *e* to the type of *x*. [...reference case...] Otherwise, the result object is direct-initialized from *e*. [Note: The conversion is ill-formed when attempting to convert an expression of class type to an inaccessible or ambiguous base class. –end note] [Note: If *T* is ``array of unknown bound of *U*`, this direct-initialization defines the type of the expression as *U*[1]. –end note].